

הצעת פרויקט כיתה יג' - פיתוח אלגוריתם יריבני למשחק מנקלה (Kalah)



שם מכללה: מכללת מקיף ה' דרכא

שם הסטודנט: אלעד ממרוד

ת"ז סטודנט: 330925546

שם מנחה: גל בר און

שם הפרויקט: משחק מנקלה (Kalah) מול יריב ממוחשב חכם

רקע תיאורטי בתחום הפרויקט

פרויקט זה מציג פיתוח מלא של משחק הלוח האסטרטגי "מנקלה" (גרסת Kalah), כולל מימוש ממשק גרפי מלא ושחקן ממוחשב (AI Agent) המסוגל להתמודד מול שחקן אנושי.

מערכת קבלת ההחלטות של ה-AI מבוססת על עקרונות תורת הגרפים: מרחב המצבים של המשחק ממודל כגרף מכון, שבו כל תצורת לוח היא צומת וכל מהלך חוקי הוא קשת המובילה למצב חדש. בניגוד לחיפוש עיוור, המודל משלב תורת המשחקים לזיהוי מצבי ניצחון.

בחינת חלופות לפתרון וניתוח SWOT

כחלק מתהליך התכנון ההנדסי ולצורך בחירת דרך הביצוע המיטבית של רכיב ה-AI, נבחנו שלוש חלופות אלגוריתמיות עיקריות. ההשוואה בוצעה באמצעות מודל SWOT כדי להבטיח עמידה בדרישות הפרויקט (זמן אמת, משאבים, רמת קושי).

חלופה א': אלגוריתם Minimax (עם Alpha-Beta Pruning)

אלגוריתם רקורסיבי הסורק את עץ המשחק לעומק רב למציאת מהלך אופטימלי מוחלט.

- **חוזקות (Strengths):** פתרון אופטימלי ובלתי מנוצח (תיאורטית); סטנדרט מקובל במשחקי לוח.
- **חולשות (Weaknesses):** סיבוכיות זמן אקספוננציאלית; עץ המשחק במנקלה מתפצל במהירות (בגלל תורות נוספים), מה שגורם לביצועים איטיים מאוד.
- **הזדמנויות (Opportunities):** יצירת שחקן "מושלם".
- **איומים (Threats):** סיכון לחריגה מדרישות זמן הריצה (Time Limit); סכנת "גלישת מחסנית" (Stack Overflow); מימוש מורכב העלול לעכב את הפרויקט.

חלופה ב': אלגוריתם חמדן נאיבי (Naive Greedy)

אלגוריתם הבוחן רק את המצב הנוכחי ובוחר את המהלך עם הרווח המיידי הגבוה ביותר.

- **חוזקות (Strengths):** קל למימוש; זמן ריצה אפסי $O(1)$; קוד פשוט.
- **חולשות (Weaknesses):** "קוצר ראייה" – נופל למלכודות פשוטות וצפוי להיות שחקן חלש מאוד.
- **הזדמנויות (Opportunities):** פיתוח מהיר מאוד של הליבה.
- **איומים (Threats):** אי-עמידה ברמה האקדמית הנדרשת (פשוט מדי); חווית משחק לא מאתגרת.

חלופה ג': אלגוריתם חמדן-יריבני (Adversarial Greedy Search) – החלופה הנבחרת

אלגוריתם היברידי המשלב חיפוש חמדני עם סימולציית שרשור תורות (Chaining) וצפי לצעד אחד של היריב.

- **חוזקות (Strengths):** מאזן אופטימלי בין ביצועים לאינטליגנציה; זמן ריצה מהיר ודטרמיניסטי; מזהה מלכודות.
- **חולשות (Weaknesses):** אינו מבטיח ניצחון מוחלט כמו Minimax (אינו רואה 10 צעדים קדימה).
- **הזדמנויות (Opportunities):** פיתוח אלגוריתם ייחודי (Custom Made) המדגים חשיבה הנדסית; אפשרות להוספת אסטרטגיות מתקדמות ("הרעבה", "אגירה").
- **איומים (Threats):** אתגר בכיוון מדויק (Tuning) של המשקולות.

טבלת השוואה וסיכום הבחירה

להלן ריכוז ההבדלים בין החלופות שנבחנו:

Adversarial Greedy (נבחר)	Naive Greedy	Minimax	קריטריון
גבוהה ואסטרטגית	נמוכה	גבוהה מאוד (מושלמת)	רמת משחק
מהיר ויציב (דטרמיניסטי)	מהיר מאוד	איטי / משתנה (תלוי עומק)	זמן ריצה
בינונית	נמוכה	גבוהה מאוד	מורכבות מימוש
גבוהה (אופטימלי)	נמוכה (פשוט מדי)	נמוכה (סיכון לחריגת משאבים)	התאמה לפרויקט

החלטה ואופן הביצוע:

לאחר שקלול הנתונים בטבלת ה-SWOT, הוחלט לבצע את הפרויקט באמצעות חלופה ג' (Adversarial Greedy).

חלופה זו נבחרה לביצוע מכיוון שהיא היחידה שמאפשרת ליצור יריב מאתגר וחכם תוך עמידה במגבלות החומרה וזמן הריצה, וכן עומדת בדרישות המקוריות של הפרויקט (ללא שימוש באלגוריתמים "אסורים" או ספריות מוכנות).

הביצוע בפועל יתבסס על מודל היברידי הכולל פונקציית הערכה משוקללת, מנגנון סימולציית שרשור (Chaining) ורכיב להכרעת מצב (Endgame Solver), כפי שמתואר בהרחבה בפרק 'הגדרת הבעיה האלגוריתמית' בהמשך המסמך.

תיאור הפרויקט

הפרויקט כולל פיתוח מערכת מלאה למשחק Kalah הכוללת:

- מנוע משחק תקני המיישם את כל חוקי Kalah
- בינה מלאכותית (AI Agent) המבוססת על חיפוש חמדני
- ממשק גרפי מלא ב-WPF
- שלוש רמות קושי שונות
- שימוש באלגוריתמים מתקדמים כגון Zobrist Hashing
- תשתית בדיקות (Unit Tests)

רקע על המשחק – מנקלה (Kalah)

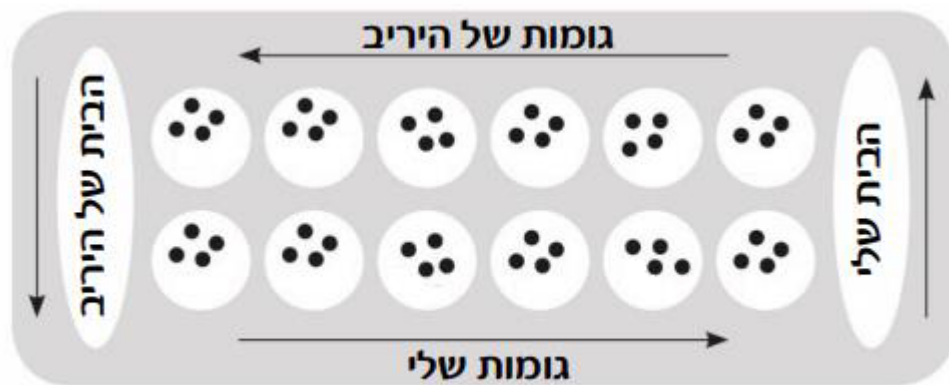
מנקלה הוא שם כולל למשפחה עתיקה של משחקי לוח אסטרטגיים שמקורם באפריקה ובמזרח התיכון. גרסת **Kalah**, שנבחרה לפרויקט זה, היא הגרסה הפופולרית ביותר כיום בעולם המערבי בשל חוקיה הברורים והמתמטיות היחסית שלה, מה שהופך אותה מתאימה במיוחד לפרויקטים בתחום בינה מלאכותית וחקר משחקים.

מבנה הלוח

לוח Kalah כולל:

- **14 גומות** בסך הכול
- 6 גומות בצד של כל שחקן
- **2 קופות (Stores / Kalahs)** – אחת לכל שחקן, ממוקמות בקצוות הלוח

סידור אבנים התחלתי קלאסי:
6 גומות לכל שחקן, ובכל אחת **4 אבנים**.



מטרת המשחק

לצבור כמה שיותר אבנים בקופה שלך עד סוף המשחק. השחקן בעל מספר האבנים הרב ביותר בקופה מנצח.

מהלך המשחק

בתורו, השחקן בוחר אחת מהגומות בצד שלו. הביצוע הוא לפי הכללים:

1. זריעה (Sowing)

השחקן אוסף את כל האבנים מהגומה שבחר ומתחיל לפזר אותן אחת אחת בכיוון נגד כיוון השעון.

הוא עובר על:

- הגומות שלו
 - הקופה שלו
 - הגומות של היריב
- אך לא על הקופה של היריב — שם אסור לו לשים אבן.

2. תור נוסף (Extra Turn)

אם האבן האחרונה נוחתת בקופה של השחקן, הוא מקבל תור נוסף.

זה חוק קריטי ומשנה-משחק, והוא חלק מרכזי בתכנון ה-AI.

3. חטיפה (Capture)

אם האבן האחרונה נופלת בגומה ריקה בצד של השחקן ובגומה שמולו בצד היריב יש אבנים, מתרחשת חטיפה:

השחקן לוקח:

- את האבן שהניח
- את כל האבנים מהגומה המולית ושם אותן ישירות בקופה שלו.

4. סוף המשחק

המשחק מסתיים כאשר:

- כל הגומות בצד של אחד השחקנים ריקות.

השחקן השני אוסף את כל האבנים שנותרו בצד שלו אל הקופה שלו. המנצח הוא מי שיש לו יותר אבנים בקופה.

הגדרת הבעיה האלגוריתמית

האתגר המרכזי בפרויקט הוא בחירת המהלך האופטימלי בכל תור משחק בתנאי זמן אמת, תוך עמידה במגבלת הדרישות האקדמיות האוסרות שימוש באלגוריתם החיפוש העמוק - Minimax Alpha-Beta Pruning או האלגוריתם ההסתברותי MCTS (Monte Carlo Tree Search). משחק המנקלה מאופיין ב־מרחב מצבים מתפשט (Explosive State Space), שבו כל תצורת לוח מובילה למספר גדול של מצבים אפשריים, ולכן חיפוש מלא בעומק אינו ישים בזמן ריצה סביר.

בנוסף, חוקי המשחק יוצרים מורכבות ייחודית: "תור נוסף" אינו רק בונס נקודתי אלא הזדמנות ליצירת שרשרת מהלכים (Momentum), ומהלכים שנראים רווחיים רגעית עלולים לחשוף את השחקן למלכודות (Traps) ולחטיפות מצד היריב.

אלגוריתם חמדן בסיסי (Naive Greedy) נוטה להיכשל בהתמודדות עם מורכבויות אלו. לפיכך, הפתרון האלגוריתמי שנבחר הוא מודל **Adversarial Greedy Search** ("חמדן יריבני"). גישה זו משדרגת את החיפוש החמדני הסטנדרטי באמצעות שני מנגנונים:

1. **סימולציית שרשור (Chaining):** האלגוריתם אינו מתייחס למהלך בודד כנקודת עצירה, אלא מבצע סימולציה רקורסיבית של כל רצף ה"תורות הנוספים" עד למיצוי התור.
2. **מזעור סיכונים:** האלגוריתם בוחן צעד אחד קדימה את התגובה החזקה ביותר של היריב, במטרה לזהות ולנטרל מהלכים שחושפים את השחקן לנזק קריטי.

שילוב זה מספק רמת משחק גבוהה המדמה חשיבה אסטרטגית, תוך שמירה על זמן ריצה קצר ודטרמיניסטי.

עקרונות אסטרטגיים ומנגנון הכרעת מצב (Game Decision Mechanics)

מערכת ה-AI מממשת לוגיקה דטרמיניסטית המבוססת על שינוי משקולות דינמי (Dynamic Weight Adjustment) ועל מעברי מצב (State Transitions) כדי להבטיח ניצחון:

1. **מנגנון הכרעת המצב (The Endgame Solver):** זהו מודול נפרד המופעל כאשר המשחק נכנס למרחב מצבים סגור.

- **הטריגר (Trigger):** האלגוריתם סופר בכל תור את סך האבנים בגומות. אם סכום האבנים (Total Stones) נמוך מערך סף מוגדר (Threshold, למשל 15), המערכת מבצעת החלפת מנוע (Context Switch).
- **הפעולה:** המערכת מפסיקה להשתמש בפונקציית ההערכה היוריסטית $h(n)$ ועוברת לחיפוש עץ מלא מסוג DFS עד לעלי העץ (מצב טרמינלי שבו הלוח ריק).
- **התוצאה:** המהלך שנבחר הוא המהלך שמחזיר את הערך המקסימלי המוחלט (Absolute Max) מבין כל נתיבי הסיום האפשריים, ובכך מבטיח ניצחון מתמטי ללא "ניחושים".

2. **מימוש אסטרטגיית "הרעבה" (Starvation Implementation):** מטרת האסטרטגיה היא למנוע מהיריב יכולת תמרון על ידי ריקון הצד שלו.

- **זיהוי:** אם סכום האבנים בצד היריב נמוך מערך קריטי (למשל, פחות מ-3 אבנים פעילות).
- **הפרקטיקה האלגוריתמית:** ה-AI מפעיל "קנס" (Penalty) על כל מהלך זריעה שחוצה את הגבול לצד היריב. בנוסף, הוא מעלה זמנית את המשקל של מהלכים המעניקים תור נוסף, כדי לשמור את השליטה ואת האבנים בצד שלו.

3. **מימוש אסטרטגיית "אגירה" (Hoarding Logic):** זוהי אסטרטגיית הגנה המופעלת על בסיס הפרש הניקוד.

- **הלוגיקה:** האלגוריתם בודק את הפרש הנקודות (ΔS).
 - אם ה-AI בפיגור משמעותי, הפרמטר w_3 (שליטה בלוח) מקבל **מקדם הכפלה חיובי**. זה גורם לפונקציית הניקוד להעדיף מצבים שבהם נשארות הרבה אבנים בצד שלי (דחיית הקץ).
 - אם ה-AI מוביל בבטחה, הפרמטר w_3 מקבל **ערך שלילי**, מה שגורם לאלגוריתם "לרצות" להיפטר מאבנים ולסיים את המשחק מהר ככל האפשר (Race to Finish).

4. **חישוב מלכודת "הבור הריק" (Empty Pit Trap Calculation):** האלגוריתם מזהה הזדמנויות לחטיפה עתידית באמצעות מנגנון ה-Lookahead.

- **הבדיקה:** עבור כל מהלך, ה-AI בודק האם הוא משאיר בור ריק בצד שלי, אשר ה"מרחק" שלו מבורות אחרים תואם למספר האבנים בהם.
- **הביצוע:** אם נמצאה התאמה (כלומר, יש בור עם K אבנים שינחת בדיוק בבור הריק), ה-AI מעניק למצב הלוח הזה בונס "פוטנציאל" (Potential Score) עוד לפני שהחטיפה בוצעה בפועל. זה גורם ל-AI להכין את המלכודת תור אחד מראש.

תהליכים עיקריים בפרוייקט

האלגוריתם לבחירת מהלך – Adversarial Greedy Search

האלגוריתם פועל בזמן אמת עבור כל מהלך אפשרי ומבצע תהליך הכרעה תלת-שלבי:

1. שלב המקסום העצמי (Simulation & Chaining):
הפקת מצבי הלוח האפשריים. אם מהלך מסתיים בקבלת "תור נוסף", האלגוריתם ממשיך לבצע סימולציה רציפה (בתוך אותו תור) כדי למצוא את המשך הרצף הטוב ביותר, עד אשר התור עובר ליריב. התוצאה היא הרווח המקסימלי שה-AI יכול להשיג ברצף זה.
2. שלב מזעור הסיכון (Opponent Lookahead):
מתוך מצב הלוח החדש, האלגוריתם מבצע סימולציה מהירה של המהלך הטוב ביותר האפשרי עבור היריב. מטרתו לזהות איזה מהלך יריב יגרום לנזק הגדול ביותר (למשל חטיפה גדולה).
3. חישוב הציון נטו (Net Score):
הציון הסופי של המהלך מחושב כהפרש בין הרווח שלי לנזק מהיריב. המהלך המוביל לציון המשוקלל הגבוה ביותר – יבחר.

מודל ההערכה (Heuristics) ורמות קושי

פונקציית הערכה:

לאחר ביצוע הסימולציות, מצב הלוח מוערך באמצעות נוסחה משוקללת:

$$h(n) = w_1 \cdot \Delta S + w_2 \cdot C + w_3 \cdot D$$

פירוט הפרמטרים:

- **ΔS – הפרש ניקוד (Net Score):** הפער המצטבר בין מספר האבנים בקופה שלי לקופת היריב. פרמטר זה מגלם בתוכו גם את הרווח שהופק משרשור תורות נוספים, ולכן מהווה את הגורם הדומיננטי בפונקציה (w_1 יקבל את המשקל הגבוה ביותר).
- **C – פוטנציאל חטיפה (Capture Threat):** האם בסיום המהלך שלי, הלוח נותר במצב שבו אני מאיים לבצע חטיפה נוספת בתור הבא.
- **D – שליטה והגנה (Board Control):** כמות האבנים שנותרה בצד הלוח שלי. המשקל (w_3) הוא דינמי ומשתנה בהתאם לשלב המשחק (Endgame Strategy) כדי למנוע מהיריב לסיים את המשחק כאשר ה-AI ביתרון, או לכפות סיום כשהניצחון מובטח.

הערה: ערכי המשקולות (w_1, w_2, w_3) ייקבעו באופן אמפירי (Empirical Tuning) במהלך שלב הבדיקות, על ידי הרצת סימולציות משחק בין גרסאות שונות של האלגוריתם.

מנגנון רמות הקושי:

השליטה ברמת האינטליגנציה תתבצע ע"י שינוי עומק החשיבה והוספת רעש:

- **רמה קלה (Easy):** האלגוריתם מדלג על שלב "בדיקת היריב" ומוסיף רעש אקראי לחישוב הניקוד. ה-AI רואה רק את הרווח המיידי ועשוי לבצע טעויות.
- **רמה בינונית (Medium):** האלגוריתם משלב רעש קל או בוחר באופן הסתברותי מבין שני המהלכים הטובים ביותר.
- **רמה קשה (Hard):** מימוש מלא של האלגוריתם, כולל שרשור תורות מלא ובדיקת תגובת יריב אופטימלית.

תורת הגרפים וייצוג המשחק

הבסיס המתמטי לפיתוח ה-AI הוא תורת הגרפים. משחק המנקלה מיוצג כגרף מכוון:

- **צמתים (Vertices):** כל מצב לוח חוקי, כלומר סידור האבנים ב-14 הגומות.
 - **קשתות (Edges):** מעבר בין מצב למצב בעקבות מהלך חוקי אחד (זריעה).
- בכל תור, ה-AI עומד בצומת אחד. באלגוריתם ה-AI אינו בוחן רק את "השכנים המידיים" של המצב הנוכחי, אלא מבצע מסלול (Path Traversal) בגרף במקרה של תורות רצופים, ובוחן צעד אחד קדימה את תגובת היריב, ובכך מקבל החלטה המבוססת על תת-גרף מקומי ולא על צומת בודד.

תיאור הטכנולוגיה

- שפת תכנות: **C#**
- סביבת עבודה: **Visual Studio 2022**
- טכנולוגיית **WPF**: GUI
- טכניקות AI:
 - Greedy Best-First Search
 - Heuristic Weighted Function
 - Zobrist Hashing
- ספריות:
 - NET.

שפות תכנות

- #C (עיקר הפיתוח)
- XAML לממשק הגרפי (WPF)

רכיבי המערכת העיקריים

מנוע המשחק (Game Engine):

מימוש מלא של כל חוקי Kalah: זריעה, חטיפה, תור נוסף, סיום משחק ועוד.

רכיב ה-AI:

יצירת מצבי לוח, חישוב תועלת, בחירת המהלך המיטבי.

מודול ההיוריסטיקה:

פונקציית הערכה עם משקולות הניתנות לכיוון.

Zobrist Hashing:

טכניקה מתקדמת לגיבוב מצבי לוח לייעול בדיקות שוויון מצבים ומניעת חישובים כפולים.

לוחות זמנים

חודש ראשון:

- תכנון מחלקות
- מחקר תיאורטי
- כתיבת מנוע משחק ראשוני (Console)

חודש שני:

- פיתוח ה-AI

- מימוש האלגוריתם החמדני

- פונקציית הערכה

חודש שלישי:

- פיתוח הממשק הגרפי

- שילוב AI בתוך ה-UI

חודש רביעי:

- בדיקות

- שיפור ביצועים

- כתיבת מסמך הגשה

מערך בדיקות (QA)

- בדיקות יחידה עבור כל חוק משחק.

- בדיקות אלגוריתם לשיפור איכות קבלת ההחלטות.

- בדיקות ביצועים להבטחת זמן תגובה מהיר.

- משחקים מול בוטים רנדומליים לבדיקת אחוזי ניצחון.

חתימת הסטודנט:



חתימת רכז המגמה:

חתימה:



חתימת המנחה:



אישור משרד החינוך: